

Feature Selection via Heuristics & Genetic Algorithms

AI-Based Programming

From raw data to structured representations

Navigating combinatorial explosion with smart search

Greedy heuristics vs. Evolutionary & Swarm algorithms

Production-level computational trade-offs

Transforming Raw Data into Learnable Signals

Definition: A feature is an individual measurable property or characteristic of a phenomenon being observed.

Data Modalities & Feature Extraction:

- **Structured Data:** Direct numerical/categorical columns (Age, Income)
- **Voice/Audio:** MFCCs capture short-term power spectrum of sound
- **Images:** CNNs extract spatial hierarchies (edges → objects)
- **Text:** Word Embeddings (Word2Vec, BERT) capture semantic meaning

The Feature Engineering Lifecycle

- **Feature Store:** Centralized data management layer in MLOps that computes, stores, and serves features consistently for training and inference
- **Feature Selection's Role:** Critical filtering step after extraction/generation, before model training
- **The Challenge:** Extraction often generates thousands of features—feeding all into a model is dangerous

The Curse of Dimensionality

Why Filter Features?

- **Performance:** Irrelevant features add noise, reducing accuracy (overfitting)
- **Computation:** More features = exponentially larger memory and training time
- **Search Space Problem:** For N features, there are 2^N possible subsets
- **Example:** If $N=30$, over 1 billion possible subsets. Exhaustive search is intractable

How Do We Score a Subset?



Filter Methods

Uses statistical tests (e.g., Pearson correlation, ANOVA) independent of the ML model.

Fast, but ignores complex feature interactions.



Embedded Methods

Selection is built into the training algorithm itself.

Examples: LASSO Regression penalties, Tree-based node splits.



Wrapper Methods

Evaluates subsets by training an actual ML model and scoring its performance.

Focus of today's lecture. Uses heuristics to explore the space efficiently.



INSTRUCTOR NOTE

"Wrapper methods give us the most direct control over our pipeline's final performance. Because they require training a model for every evaluation, we have to be extremely smart about how we pick which subsets to test."

Why Wrapper Methods?

Wrapper methods give us the most direct control over pipeline's final performance.

- Train model for every evaluation—computationally expensive
- Must be smart about which subsets to test
- Uses heuristics to explore 2^N space efficiently
- Provides feedback loop: Data $\rightarrow$ Algorithm $\rightarrow$ Score $\rightarrow$ Refinement

Heuristic Search: Forward and Backward Strategies

Heuristic Definition: An algorithm designed to solve a problem faster by trading mathematical optimality for speed.

➔ Forward Selection

- 1 Start with an empty feature set.
2. Train models adding one feature at a time.
3. Keep the feature that yields the highest accuracy gain.
4. Repeat until accuracy plateaus.

↪ Backward Elimination

- 1 Start with all features.
2. Remove one feature at a time.
3. Permanently drop the feature whose removal causes the smallest drop in performance.



INSTRUCTOR NOTE

"Greedy algorithms make the locally optimal choice at each stage. They are straightforward, deterministic, and much faster than exhaustive search. Notice how they build or prune trees step-by-step."

```
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.linear_model import LogisticRegression
```

```
# 1. Define the base estimator (Wrapper model)
model = LogisticRegression()
```

```
# 2. Forward Selection
sfs_forward = SequentialFeatureSelector(
    estimator=model,
    n_features_to_select=5,
    direction='forward',
    cv=3
).fit(X_train, y_train)
```

```
# 3. Backward Elimination
```

```
sfs_backward = SequentialFeatureSelector(
    estimator=model,
    n_features_to_select=5,
    direction='backward',
    cv=3
).fit(X_train, y_train)
```

```
# Get the boolean mask of selected features
print("Forward Selected:", sfs_forward.get_support())
```

SequentialFeatureSelector in Scikit-Learn



The Flaw in Greedy Search

The Nesting Effect & Local Optima

- **Nesting Effect:** Forward—once added, never removed. Backward—once removed, never added back
- **Problem:** Feature A may be highly predictive alone, but redundant when B and C are together
- **Solution:** Stochastic, population-based metaheuristics (Evolutionary & Swarm)

Genetic Algorithms Overview

Inspired by Darwinian natural selection

- **Gene:** Single feature (0 or 1)
- **Chromosome:** Feature subset [1, 0, 0, 1]
- **Population:** Group of different subsets
- **Generation:** One search iteration

GA Operations: Evolution in Action

- **Selection:** Fittest individuals chosen to reproduce
- **Crossover:** Swapping segments of parent arrays to create offspring
- **Mutation:** Randomly flip bits ($0 \leftrightarrow 1$) to maintain diversity and escape local optima
- **Lifecycle:** Initialize → Calculate Fitness → Select → Crossover → Mutate → Repeat

Defining the Fitness Function

Balancing accuracy and complexity—we want high accuracy with few features for fast systems.

$$\text{Fitness} = (\alpha \cdot \text{Accuracy}) + \beta \cdot (1 - |S| / D)$$
$$\beta = (1 - \alpha)$$

α and β are weights balancing predictive performance vs. pipeline sparsity

$$Fitness = \alpha \times Error + (1 - \alpha) \times \frac{|S|}{D}$$

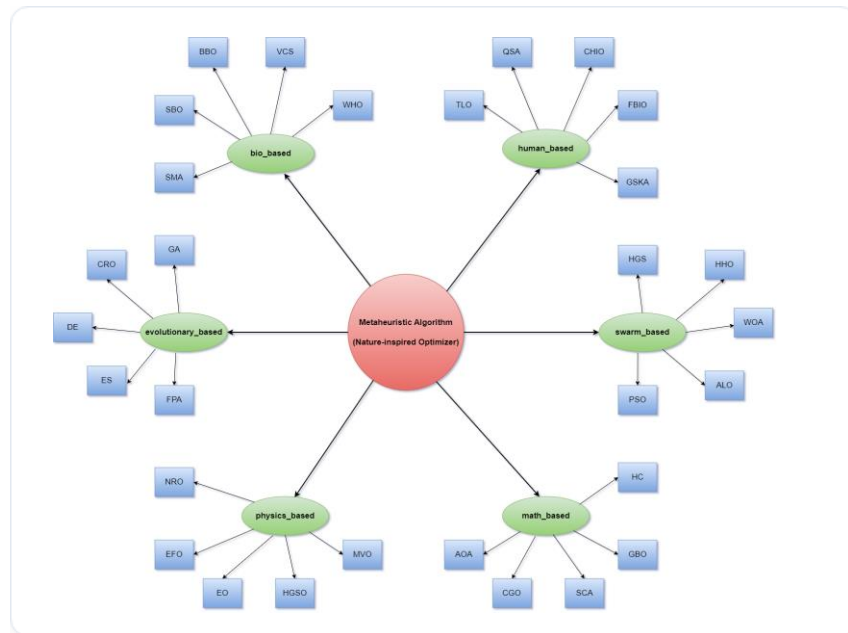
where

- Error = classification error
- $|S|$ = number of selected features
- D = total features
- α controls the trade-off

The MEALPY Ecosystem: Families of Metaheuristics

MEALPY organizes its optimization algorithms into distinct "families" based on their real-world inspiration. Depending on your feature space, different families may converge faster:

- **evolutionary_based**: Simulates natural selection (e.g., GA: Genetic Algorithm, DE: Differential Evolution).
- **swarm_based**: Simulates collective behavior of animals (e.g., PSO: Particle Swarm, WOA: Whale Optimization).
- **physics_based**: Mimics physical laws and thermodynamics (e.g., CDO, SA: Simulated Annealing).
- **bio_based** / **human_based**: Inspired by biological processes or human social behaviors.
- **math_based** / **music_based**: Inspired by math functions or musical harmony.



```
import numpy as np
from mealpy.swarm_based import PSO
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
```

```
# fitness function
def fitness_function(solution):

    solution = np.array(solution) > 0.5
    selected_features = np.where(solution)[0]

    if len(selected_features) == 0:
        return 1

    X_selected = X[:, selected_features]

    clf = KNeighborsClassifier(n_neighbors=5)

    score = cross_val_score(clf, X_selected, y, cv=5).mean()

    error = 1 - score
    feature_ratio = len(selected_features) / X.shape[1]

    fitness = 0.99 * error + 0.01 * feature_ratio

    return fitness
```

MEALPY-Based Feature Selection (Part 1)

MEALPY-Based Feature Selection (Part 1)

```
problem = {  
    "fit_func": fitness_function,  
    "lb": [0]*X.shape[1],  
    "ub": [1]*X.shape[1],  
    "minmax": "min",  
}
```

```
model = PSO.BasePSO(  
    epoch=100,  
    pop_size=30  
)  
  
best_position, best_fitness = model.solve(problem)
```

```
selected_features = np.where(np.array(best_position) > 0.5)[0]  
  
X_selected = X[:, selected_features]
```

Productionizing: Computational Trade-offs

The Bottleneck:

Population of 50 over 100 epochs = 5,000 model trainings

System Architecture Solutions:

- **Parallelization:** Distribute fitness evaluations across clusters (`n_workers` in MEALPY, Spark)
- **Memoization:** Store fitness scores in Redis cache to bypass redundant training

Engineering Metaheuristics for Production

- **Proxy Models:** Use lightweight model (Decision Tree, Logistic Regression) during search phase
- Train heavy model (Transformer, Deep NN) only once on final winning subset
- **Trade-off:** Slow training phase, blazing fast inference pipeline
- **Result:** Optimal feature subset enables real-time predictions in production

Key Takeaways

- Feature selection navigates 2^N combinatorial space
- Greedy algorithms fast but get trapped in local optima
- Genetic & metaheuristic algorithms explore globally
- MEALPY provides 200+ optimization algorithms
- Production requires parallelization, caching, proxy models